Guarded Recursive Self-Improvement: Bounding Creeping Degradation and Reward Hacking in Autonomous Agent Loops

Abdelkabir Ouadoukou*

November 2026
Technical report (Study 2). Not peer reviewed.

Abstract

Agents that edit their own code can improve, but the usual loop ("propose a patch, keep it if a score goes up") fails in three ways: many individually harmless-looking patches add up to a regression (*creeping degradation*), the selection data is reused until the score no longer means anything (*adaptive overfitting*), and the agent finds ways to satisfy the checker instead of the task (*reward hacking*). Study 1 added a second problem: a judge that is too strict rejects almost everything. We treat self-improvement as a control problem and describe SI-Loop v2, a kernel that sits between a proposing agent and a sealed verifier. Its parts are: zero-trust git worktrees with a seven-layer patch defence; a paired non-inferiority gate applied in two stages (a screen on batch A with anytime-valid sequential stopping, then a strict test on an independent batch B) that removes the winner's curse; a frozen cycle-0 anchor with a finite *margin budget* that only proven gain can refill; a held-out problem family that selection never touches; a three-tier verifier with honeypots and a measured mutation kill rate; a hash-chained ledger with circuit breakers; and pre-registered ablation arms (A0 frozen improver, A1 full recursion, A2 random patches, A3 compute-matched best-of-N) to test whether recursion adds anything. On a simulated null patch, the two-stage judge accepts 0.9% of patches at its default threshold, against 4.3% for a single stage, with a power loss that shrinks to about two points for a true gain of eight points. The verifier's tiers kill 86.7% of 248 injected faults (99.5% after removing survivors that pass an extra-large fuzz run and are likely equivalent). In a pre-registered pilot on a *synthetic* world with 8 seeds, we report the ablation results as they came out (Section 5.4), including what the pilot cannot show. We also provide a reference implementation with hidden pytest cases and an LLM meta-agent interface. We do *not* claim recursive self-improvement of language models; we claim a measurable, auditable harness in which such a claim could be tested.

1 Introduction

Systems that rewrite their own code are no longer hypothetical: the Darwin Gödel Machine validates self-modifications empirically [13], hyperagents make the improvement procedure itself editable [14], and the Huxley-Gödel Machine argues that a parent should be judged by the productivity of its descendants, not its own score [11]; the classical formulation goes back to Gödel machines [9]. Surveys cover the wider landscape [2]. These systems mostly rely on one signal (a benchmark score) and one rule (keep the patch if the score rises). That is where the failures start.

*Independent researcher. Contact: abdelkabir.ouadoukou@gmail.com. This work was carried out with substantial AI assistance; see the disclosure at the end of the paper.

Failure modes of the simple loop.

- **Creeping degradation.** A gate that accepts any patch that is "not significantly worse" also accepts a patch that is slightly worse. Each step passes; a hundred steps do not. Rewards for "smaller" or "cheaper" code make this path attractive.
- **Adaptive overfitting.** Selecting among many patches on the same distribution inflates the measured gain (winner's curse), and every reported number leaks information [5].
- **Reward hacking.** Proxy objectives are exploited when optimised hard [3, 6], and self-play judges can be convinced [15]. For code agents this includes reading the tests, writing the result file, or special-casing the checker.
- **Over-conservative judging.** In Study 1 the guarded judge rejected about 73% of rounds and the arm without a judge won. Safety that blocks all progress is also a failure.
- **Unmeasured recursion.** A rising curve does not show that improving the *improver* helped. Without a frozen-improver control, compute-matched sampling and random-patch baselines, "recursive" is a label.

Approach. Prompting the agent to be careful is not a control. We instead put a kernel in front of the agent that the agent cannot modify, make every acceptance a calibrated statistical decision on fresh data, bound the total slack the system may spend, keep one family of problems completely out of the selection loop to measure generalisation, and measure the whole thing against pre-registered controls.

Contributions. (1) A two-stage, anytime-valid acceptance pipeline and its false-accept calibration (Sections 3.2, 5.1). (2) A margin budget and anchor audit that bound creeping degradation (Section 3.4). (3) A verifier design with three tiers, honeypots and a measured kill rate (Sections 3.6, 5.2). (4) A yield test for the improver, clade parent selection and an operator bandit (Section 3.7). (5) A tamper-evident ledger and circuit breakers for long runs (Section 3.8). (6) A pre-registered ablation runner and a reference implementation with an LLM interface (Sections 4, 5.4). Every component has prior art; our claim is the integration and the measurements, reported with their limits.

2 Threat Model and Design Principles

Adversary. The meta-agent is untrusted: it may be buggy, over-optimising or actively trying to raise its score by illegitimate means. It has read access to its own worktree and a token budget. It can submit one structured patch per cycle. We do *not* defend against an adversary with code execution outside the sandbox, with access to the verifier's secret, or against kernel bugs; local mode is not a security boundary.

Principles. (P1) *Immutable kernel*: the code that judges is not in the code that is judged; a digest check aborts the run on any change. (P2) *Agents propose, the kernel disposes*: no direct writes; every change is a validated, atomic, reversible patch. (P3) *Fresh data for every decision*: problem ids are HMACs of a secret, per split and per cycle. (P4) *Coarse feedback*: the agent learns only accept / stepping_stone / reject and a brief error type. (P5) *Bounded slack*: every non-proven accept spends from a finite budget; audits compare against a frozen reference. (P6) *Measure recursion against controls*, not against nothing.

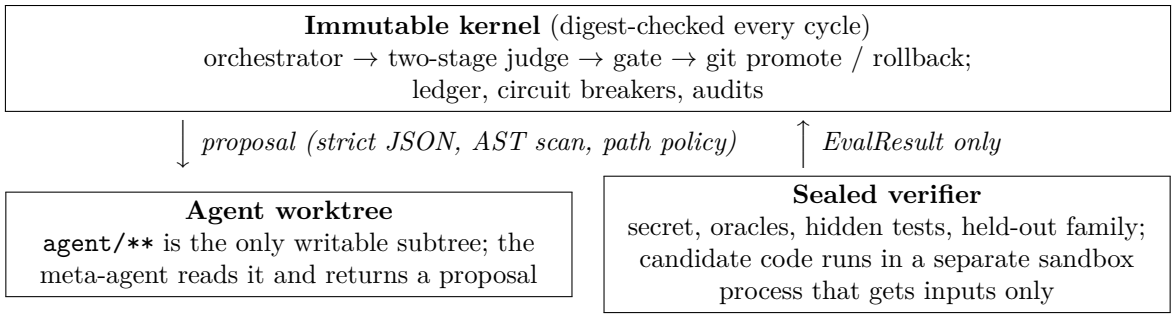


Figure 1: Trust boundaries. The agent never sees hidden tests, expected values, problem content, per-family numbers, counts, audit or held-out results.

Table 1: The seven layers applied to every patch.

#	Layer	What it rejects
1	Strict parse	unknown keys, wrong types, more than 8 ops, oversize payloads, bad tags
2	Path policy	anything outside `agent/`, traversal, dotfiles, `conftest.py`, other suffixes
3	AST scan	forbidden imports (`socket`, `subprocess`, `ctypes`, ...), `eval`/`exec`/`open`, dunder escapes, dangerous `os.*`
4	Test ratchet	removed tests; `skip`, `xfail`, `importorskip`
5	Atomic exact apply	ambiguous symbols; any failing op leaves the tree untouched
6	Diff audit	committed files differing from the files named in the proposal
7	Sandboxed self-tests	failing public tests, timeouts, memory blow-up

3 The SI-Loop v2 Architecture

3.1 Zero-trust worktrees and the seven-layer patch defence

Each cycle creates a git worktree from the chosen parent. The meta-agent returns a proposal; the kernel checks it through seven layers before a single test runs (Table 1). A proposal is a list of at most eight *hunks*; each hunk is the unit of credit assignment. The static scan is a first filter, not a security boundary: real isolation comes from the sandbox process (rlimits, scrubbed environment, no network, process-group kill; a container mode adds read-only root and `-network none`).

3.2 The paired non-inferiority gate

Parent and candidate run on the same fresh problems. Let b be the number of problems the patch fixed, c the number it broke, n the number of problems, $\delta = (b-c)/n$, $\mathrm{se} = \sqrt{b+c-(b-c)^2/n}\,/n$ and $\ell = \delta - z\,\mathrm{se}$ with $z = 1.645$ (a normal approximation; the one-sided lower bound used for non-inferiority [1]). With a flat prior on the fix probability among discordant pairs,

$$p_{\text{better}} = P\big(\mathrm{Bin}(b+c+1, \tfrac{1}{2}) \le b\big).$$

A patch is accepted if $\ell \ge -\varepsilon$, $\delta > 0$ and $p_{\text{better}} \ge p_{\min}$, or (utility path) if it is non-inferior and clearly smaller or cheaper. A non-inferior, behaviourally novel patch becomes a *stepping stone* (archived, not promoted); novelty can never rescue a regression. The threshold $p_{\min}$ is chosen by A/A calibration for the batch size at hand (Section 5.1).

3.3 Two stages, sequential stopping and online FDR

Screen then confirm. Batch A (lenient, $p_{\text{screen}} = 0.80$) is evaluated in chunks. A candidate that passes is evaluated on an *independent* batch B, and the strict gate is applied to batch B alone. Because B was not used to select the candidate, the winner's curse does not apply to it; because both stages must pass, the false-accept probabilities multiply.

Anytime-valid early stopping. After each chunk of A we update the discordant counts (b, c). For harm we use the mixture e-value

$$E_{\downarrow} = \frac{B(c+1, b+1)\sum_{k \le c}\binom{b+c+1}{k}}{1}, \qquad \text{stop if } E_{\downarrow} \ge 1/\alpha_{\text{harm}},$$

the mirror of the improvement e-value $E_{\uparrow} = B(b+1, c+1)\sum_{k \le b}\binom{b+c+1}{k}$, which is a mixture over $q \sim U(\frac{1}{2}, 1)$ of the likelihood ratio $q^b(1-q)^c/(1/2)^{b+c}$ and hence an e-value under every null with fix probability $\le \frac{1}{2}$. By Ville's inequality [8], stopping at any data-dependent time keeps the error bound. For futility we use Wald's SPRT [10] with alternative $p_1 = \frac{1}{2} + \theta$. A stop only drops a candidate early; it cannot create an accept, because acceptance happens on B.

Online FDR. Over thousands of cycles the number of accepts under the null grows. Confirmed accepts are optionally passed to e-LOND [12], which controls the false discovery rate under arbitrary dependence; we run it in monitor mode (logged) or enforce mode (blocking).

3.4 Anchor, margin budget and audits

The champion at cycle 0 is frozen as the *anchor*. Accepts that are not backed by a proven gain (the utility path) spend from a finite *margin budget* B_0. The charge of an accept with lower bound ℓ_i is $c_i = \max(\varepsilon_{\min}, -\ell_i)$ (the amount of regression its bound could hide, with a floor). The budget is refilled only by a *new high-water mark* of proven gain over the anchor, measured in audits on fresh batches. Every K cycles an audit compares the champion with the anchor and with its own historical peak and rolls back on regression.

Proposition 1 (Budget bound)**.** *Suppose utility-path accepts $i = 1, \dots, k$ have true paired changes δ_i and lower bounds ℓ_i with $P(\delta_i \ge \ell_i) \ge 1 - \alpha$, and charges $c_i \ge \max(0, -\ell_i)$. On the event that all bounds hold (probability $\ge 1 - k\alpha$ by a union bound), the total regression introduced by these accepts satisfies $\sum_i \max(0, -\delta_i) \le \sum_i c_i \le B_0 + R$, where R is the total refill, and R is at most the proven gain over the anchor.*

Sketch. On the event, $-\delta_i \le -\ell_i \le c_i$ whenever $\delta_i < 0$. Spending cannot exceed the balance, so $\sum c_i \le B_0 + R$, and refills equal increments of a high-water mark of lower bounds on gain over the anchor. □

The proposition is deliberately modest: it bounds the sum of step losses (not the net drift), assumes valid per-step bounds (ours are normal approximations), and ignores correlation between steps. The anchor audit is the backstop for exactly these gaps.

3.5 Held-out family and noisy audits

One problem family is drawn *only* by the audit split. It is never used for selection, confirmation, credit, anchor or yield batches. The cross-family gap is the selection gain minus the held-out gain

of the champion against the anchor. Audit decisions that read reusable data are made through a Thresholdout-style noisy comparison [5], and results are written to the ledger, never to the agent. A large gap pauses the run.

3.6 Sealed verifier: three tiers, honeypots, mutation testing

Every problem is checked by three tiers: small-scope exhaustive inputs, random large-scope fuzzing, and metamorphic invariance relations (for example, adding spaces around operators must not change an arithmetic result). The oracle and the comparison run in the judge; candidate code runs in a sandbox worker. Honeypots are impossible problems with junk expectations that exist only in decoy files; passing one, opening a decoy path, or emitting a canary sets a security flag and an automatic reject, and the commit is quarantined. To measure the verifier itself we apply AST mutations (comparison swaps, operator swaps, constant changes, deleted statements, `return None`) to known-correct reference solutions and count how many mutants the tiers reject [4, 7]. Survivors that also survive an extra-large fuzz run are counted as likely equivalent mutants.

3.7 Yield testing, clades and the operator bandit

A patch that edits the improver (`agent/meta/**`) changes how future patches are made, so a solver score cannot judge it. We evaluate its *yield*: the gain in pass rate after K inner cycles on a fresh batch, from identical start states and seeds, for the candidate improver, its parent, and the frozen reference improver M0. It is accepted iff the bootstrap lower bound of the paired yield difference to the parent is positive and the yield is not worse than M0 by more than a tolerance. Mixed-scope patches are rejected. Parents are chosen by Thompson sampling on discounted descendant successes (clade productivity, after 11), not by their own score. An operator bandit tracks an exponential moving average of reward per hunk tag.

3.8 Long-run autonomy

The ledger is append-only and hash-chained: $h_t = \mathrm{SHA256}(h_{t-1} \| \mathrm{canon}(r_t))$, so any edit of history is detected on verification. Circuit breakers trip on: accept-rate anomalies (binomial tail against a pre-registered rate), LOC explosions, cost spikes, repeated sandbox-escape attempts, security flags and repeated audit regressions; a trip pauses or latches a HALT file. A heartbeat and an external watchdog cover a silent crash; a model and kernel fingerprint is logged and a change halts the run because it invalidates calibration. Orphan worktrees and stale branches are garbage-collected.

4 Reference Implementation

Two reviewer concerns about earlier drafts were that the verifier and the meta-agent were only interfaces. The repository now contains a complete reference pair.

Task suite and hidden pytest. `study2/` defines 15 tasks in three families: string (6), algorithmic (6) and a held-out parsing family (3). Each has a public specification (a docstring), a judge-side oracle and a case generator that mixes ordinary and edge-case inputs. The seed agent has about half of the functions buggy or missing. For each evaluation the judge derives the tasks from HMAC-based ids, copies the candidate's `agent/` into a sandbox process that receives *inputs only*, collects the outputs, writes a hidden pytest file with expected and actual values, and runs pytest in its own process. A problem passes iff its pytest test passes on all six cases. Because

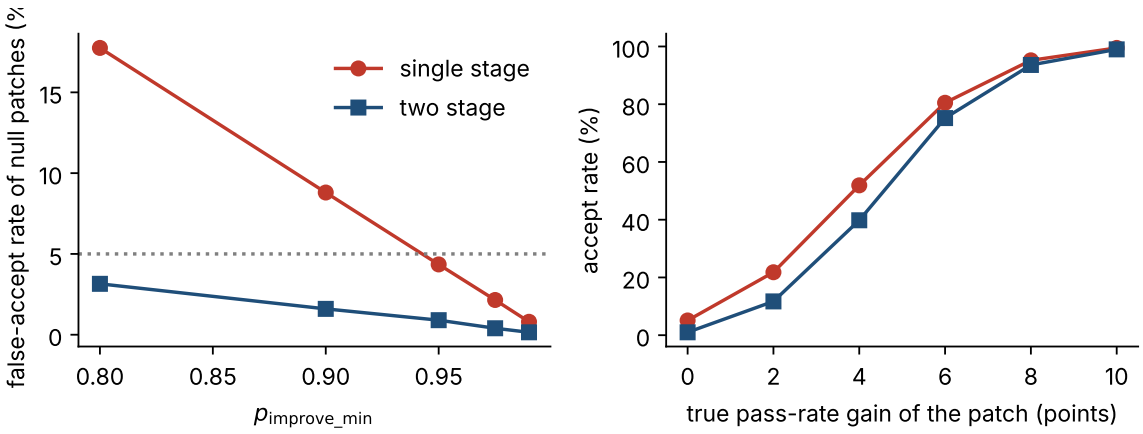


Figure 2: Left: false-accept rate of null patches against $p_{\text{improve_min}}$. Right: accept rate against the true gain (at $p_{\text{improve_min}} = 0.95$).

candidate code never shares a process with pytest or an oracle, it cannot patch the test runner or read an expected value; a forged result without the per-run nonce is ignored, and a decoy file tripwire flags attempts to read "hidden" paths. A separate self-check confirms that a library of oracle solutions passes 100% (no false rejects).

LLM meta-agent. `LLMMetaAgent` builds a prompt from the agent's own source and untrusted lessons, asks for a JSON proposal, parses it with the kernel's strict `parse_proposal`, and retries malformed replies with the error text; all calls are charged to the cycle's token budget. Clients exist for any OpenAI-compatible endpoint (a local Ollama, vLLM, llama.cpp or LM Studio server, or OpenAI) and for the Anthropic API, plus a scripted offline client used for tests and demos. `run_study2.py` wires everything together (`check`, `demo`, `run`, `report`).

5 Evaluation

We separate what is measured from what is only demonstrated. Sections 5.1 and 5.2 are measurements of components. Section 5.3 demonstrates the pipeline with a scripted proposer. Section 5.4 is a pre-registered pilot on a synthetic world.

5.1 False-accept calibration and power

We simulate A/A tests: both versions have the same true pass rate $p = 0.5$, with independent run-to-run noise ($\sigma = 0.1$ on the latent scale), $n = 200$ problems per batch and 2000 trials per setting (power uses 1000). At $p_{\min} = 0.95$ the single-stage gate accepts 4.3% of null patches and the two-stage judge 0.9%. Figure 2 and Table 2 show the full curves. The price is power: for a patch that truly adds 2, 4, 6, 8 points of pass rate, the single stage accepts 22%, 52%, 80%, 95% and the two-stage judge 12%, 40%, 75%, 94%. Small true gains are therefore the ones that are lost, which is the intended trade: they are also the ones most easily produced by noise. The noise model is an assumption; estimate the real run-to-run noise from repeated evaluations of an unchanged agent before trusting these numbers.

Table 2: A/A false-accept rate ($n = 200$, $p = 0.5$, $\sigma = 0.1$).

$p_{\text{improve_min}}$	single stage	two stage
0.800	17.8%	3.1%
0.900	8.8%	1.6%
0.950	4.3%	0.9%
0.975	2.1%	0.4%
0.990	0.8%	0.1%

5.2 Verifier quality

On the synthetic world's nine sealed task kinds we generated 248 mutants of the reference solutions; the three tiers killed 215: a raw kill rate of 86.7%. Of the survivors, 32 also passed an extra-large fuzz run and are treated as likely equivalent mutants (for example, edits to code paths that the sampled parameters never reach), giving 99.5%. The "equivalent" label is a heuristic, not a proof, so the raw rate is the conservative figure. Kills by tier were small-scope 203, fuzz 211, metamorphic 23; mutants killed *only* by one tier were 4, 12 and 0 respectively, which is each tier's marginal value. In this suite the metamorphic tier killed no mutant that the other tiers missed; its value here is limited to the arithmetic kinds, where it targets tokenisation-style bugs that small-scope tests can miss. We keep it because a kill rate on injected faults does not measure the faults we did not inject. The correct reference solutions failed 0 times (they must never fail), and the planted snooping solver was detected: yes. The adjusted rate is evidence about the injected-fault class only; it does not bound the probability of a verifier gap an LLM would find.

5.3 End-to-end pipeline run (scripted proposer)

`run_study2.py demo` runs 14 cycles of the reference implementation with a *scripted* proposer (a list of fixes, one deliberate regression, one deliberately malformed reply); it is not a language model and demonstrates the machinery only. Of the 14 cycles, 7 were accepted, 2 were stepping stones and 5 were rejected; the hash chain of the ledger was verified. On fresh batches the champion's pass rate on the selection families rose from 0.33 (anchor) to 0.77 (52 problems fixed, 0 broken out of 120), while the held-out family stayed at $0.36 \to 0.36$. The cross-family gap of 0.43 raised the generalisation alarm, as it should: the scripted proposer fixes only functions in the selection families. The deliberate regression (a function returning 0) was stopped by the sandboxed self-tests, and the malformed reply was retried.

5.4 Pre-registered ablation pilot

Design. The unit of analysis is the run (seed). Arms: A0 frozen improver (solver patches only), A1 full recursion (solver and improver, improver judged by yield tests), A2 random-patch control, A3 compute-matched best-of-N (every proposal is made from the anchor and the champion is the best single patch). Each arm runs 12 cycles on 8 seeds in the smoke profile; final evaluation uses fresh batches shared by every arm and seed. Hypotheses are fixed before the runs: H1 A1>A0 on final selection gain; H2 A0>A2; H3 A0>A3 (one-sided paired sign-flip tests, Holm-corrected over H1–H3); H4 A1−A0 on held-out gain ≥ -0.02. The pre-registration (hash b36d851de8d8…) is locked to the kernel digest; the runner refuses to start if either changes. With 8 seeds the smallest attainable one-sided p-value is 0.0039. The world is synthetic: the "meta-agent" is a seeded mutation engine, so A1 tests whether tuning the *policy* of such an engine helps. It tests the harness, not an LLM.

Table 3: Arms, mean ± SD over seeds. Gain is the final pass rate minus the anchor's, on fresh batches shared by all arms. Tokens include yield tests.

Arm	seeds	gain, selection families	gain, held-out	accepts	tokens
A0	8	$+0.178 \pm 0.129$	+0.000	1.5	12,534
A1	8	$+0.302 \pm 0.220$	+0.000	1.8	29,098
A2	8	-0.017 ± 0.018	+0.000	2.6	11,776
A3	8	$+0.242 \pm 0.122$	+0.000	1.0	12,429

Table 4: Pre-registered contrasts (paired by seed; one-sided sign-flip p; Holm over H1–H3).

Hyp.	contrast	pairs	mean diff.	95% bootstrap CI	p (one-sided)	Holm p
H1	A1−A0	8	+0.124	$[-0.076, +0.319]$	0.1523	0.3047
H2	A0−A2	8	+0.195	$[+0.120, +0.272]$	0.0039	0.0117
H3	A0−A3	8	−0.064	$[-0.177, +0.066]$	0.8438	0.8438

Results. Table 3 and Table 4 are generated from the raw per-run files. No run halted, tripped a security flag or broke the ledger (0 such runs). The mean final gain on the selection families over the frozen cycle-0 anchor was +0.178 (A0), +0.302 (A1), -0.017 (A2) and +0.242 (A3).

- **H2 supported** (A0−A2 = 0.195, Holm $p = 0.0117$). Random patches behind the same gate did not improve the agent (-0.017), while informed proposals behind the same gate did. The gain is therefore not an artefact of the gate or of the evaluation. The gate also kept the random arm from degrading: its mean loss was small, but not zero (below).
- **H1 not supported** (A1−A0 = 0.124, 95% CI [-0.076, 0.319], Holm $p = 0.3047$). The point estimate favours A1, but the interval includes zero. More importantly, *no improver patch was accepted in any A1 run*: 13 improver patches were proposed across the eight runs and none was accepted. A1 and A0 therefore ran the same improver policy except that A1 spent about 15% of its proposals on (rejected) policy edits. The positive difference reflects run-to-run variation of the proposal stream (A1's per-seed gains are bimodal), not recursion. This pilot cannot say anything about whether improving the improver helps.
- **H3 not supported** (A0−A3 = -0.064, Holm $p = 0.8438$). Compute-matched best-of-N from the anchor (+0.242) was at least as good as iterating and accumulating (+0.178). In this small world each useful fix is independent, so iteration adds nothing over sampling. A1 used about 2.3× the tokens of the other arms because of yield tests (Table 3); A1 vs. A3 was not pre-registered and is not tested.
- **H4 is uninformative.** The held-out gain was +0.000 in every arm because the toy improver never edits the interpreter family; the non-inferiority margin held trivially. The held-out audit machinery is exercised in the end-to-end run (Section 5.3), but this ablation provides no evidence about cross-family overfitting.

The random arm and the margin budget. A2 accepted 2.6 patches per run on average. Its final gain ranged from −0.052 to 0.000 (mean -0.017); the margin spent per run ranged from 0.005 to 0.020 against a budget of 0.03. The worst run lost 0.052 on the final batch while having spent only 0.005. That is a counter-example to reading the budget as a hard bound on true loss: the charge is a lower-bound-based estimate, the final batch (96 problems) is itself noisy (standard error

roughly 0.02), and the proposition holds only when the per-step bounds hold. What the data do support is narrower: with the budget and floor in place, a policy that proposes random edits lost about two points on average and no more than five in the worst of eight runs, and never gained.

What this pilot does and does not show. It shows that the harness runs end to end under pre-registration, that the gate separates informed from random search (H2), and that the controls (A2, A3) are needed: without A3, the gain of A0 or A1 would have been read as evidence for iteration. It does not show recursive self-improvement; the one test of it (H1) is empty because the improver never changed. With 8 seeds the smallest attainable one-sided p-value is 0.0039, so only large, consistent effects could be detected. A real study needs more seeds, a world in which improver edits can matter, and an LLM meta-agent.

6 Discussion and Limitations

Adaptive overfitting. The two-stage design removes the winner's curse for each decision, and fresh HMAC batches prevent reuse, but over many cycles the agent can still adapt to the *distribution* (families, edge-case style) through accepted code. Coarse feedback limits leakage rather than eliminating it; the held-out family is the detector, and it only detects generalisation failure across families we thought of. The scripted run shows the alarm firing; whether an LLM would overfit is an open empirical question.

Verifier quality. Mutation kill rate measures sensitivity to simple injected faults. It says little about semantic loopholes (an under-specified oracle, a missing edge-case family). Honeypots catch curiosity, not stealth. Every verifier needs red-teaming by an adversarial model before a claim of "no reward hacking" is made.

Margin budget. The proposition is conditional and bounds a sum, not drift; underestimated charges are possible. The budget makes creep expensive and visible, and the audit makes it reversible, but neither makes it impossible.

Synthetic mutation engine versus real LLM meta-agents. Our ablation uses a simulated improver because it is deterministic, free and fast; its dynamics (a handful of operators, a library of fixes) are far from an LLM's. The step from simulation to a model is where most of the uncertainty lies: proposals become much more varied, token costs matter, failure modes include prompt injection through source comments, and run-to-run noise rises, so batch sizes and $p_{\min}$ must be recalibrated. The kernel does not change; the calibration, the budgets, the number of seeds and the cost of yield tests do. We provide the interface (`LLMMetaAgent`) and a local-model path, but report no LLM results here.

Other limits. Container mode is untested; the local sandbox is not a security boundary. Cost is counted in LOC and tokens, not deterministic instructions. The pilot has few seeds and a small world. The judge's normal-approximation lower bound is not exact for small discordant counts. Several references in `references.bib` are marked FROM-KNOWLEDGE and must be checked before submission.

7 Conclusion and Reproducibility

Guarding self-improvement is mostly bookkeeping: who may write, which data decides, how much slack remains, what was measured against which control. SI-Loop v2 puts that bookkeeping in an immutable kernel, calibrates its acceptance rule, and ships a pre-registered harness for asking whether recursion helps. The next step is the one this paper does not take: running it with a real LLM meta-agent, with the calibration and seeds this setting requires.

Reproducibility. Code and data: https://github.com/abdelkabirouadoukou/si-loop. Tests: `python -m pytest -q tests`. Verifier self-check: `python run_study2.py check`. Demo: `python run_study2.py demo`. Calibration: `python results/study2/make_calibration.py`. Verifier quality: `python -m kernel.mutation`. Ablations: `python run_ablations.py prereg|run|analyze`. All numbers and tables in Section 5 are generated by `results/study2/make_paper_numbers.py` from the raw result files in `results/study2/`. Study 1 is frozen and unchanged; `si_loop_toy.py` is pinned by hash in `results/config_hashes.txt`.

AI assistance disclosure. The code, the experiments' scripts and the drafting of this report were produced with substantial assistance from an AI system (Anthropic's Claude), under the author's direction. The author decided the study design and is responsible for the content. Numerical values come from generated files; references marked FROM-KNOWLEDGE in the bibliography were not re-fetched.

References

[1] William C. Blackwelder. "proving the null hypothesis" in clinical trials. *Controlled Clinical Trials*, 3(4):345–353, 1982.

[2] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026.

[3] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02743.

[4] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[5] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248): 636–638, 2015.

[6] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2210.10760.

[7] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[8] Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023.

[9] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. *Artificial General Intelligence (Cognitive Technologies), Springer*, 2007. arXiv:cs/0309048.

[10] Abraham Wald. Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2):117–186, 1945.

[11] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.21614.

[12] Ziyu Xu and Aaditya Ramdas. Online multiple testing with e-values. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238 of *PMLR*, 2024.

[13] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[14] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026.

[15] Chenyu Zhou. More convincing, not more correct: Self-play reward hacking of reference-free LLM judges. *arXiv preprint arXiv:2607.05904*, 2026.